

SANCHARI

Smart AI-Powered Mobility & Pooling Ecosystem

"Share Seats. Save Costs. Travel Smarter."

GITHUB_URL : [MAKINEEDI05/Sanchari-](https://github.com/MAKINEEDI05/Sanchari-)

Table of Contents

1. Introduction
2. Project Description
3. Problem Statement
4. Proposed Solution
5. Functional Requirements
6. Platform Features
7. AI-Powered Optimization Engine
8. Security & Safety Infrastructure
9. Architecture (Technical & Non-Technical)
10. Real-Time Distributed Communication Architecture
11. Environmental & Social Impact
12. Future Scalability & Expansion
13. Conclusion

1. Introduction

Sanchari is a scalable AI-powered ride-pooling and mobility platform designed to solve modern urban commuting challenges through intelligent shared transportation.

Unlike traditional ride-hailing applications that focus primarily on instant point-to-point rides, Sanchari is built around scheduled car-pooling, trip-pooling, and parcel-pooling to maximize vehicle utilization, reduce fuel consumption, and lower transportation costs.

Core Mobility Services

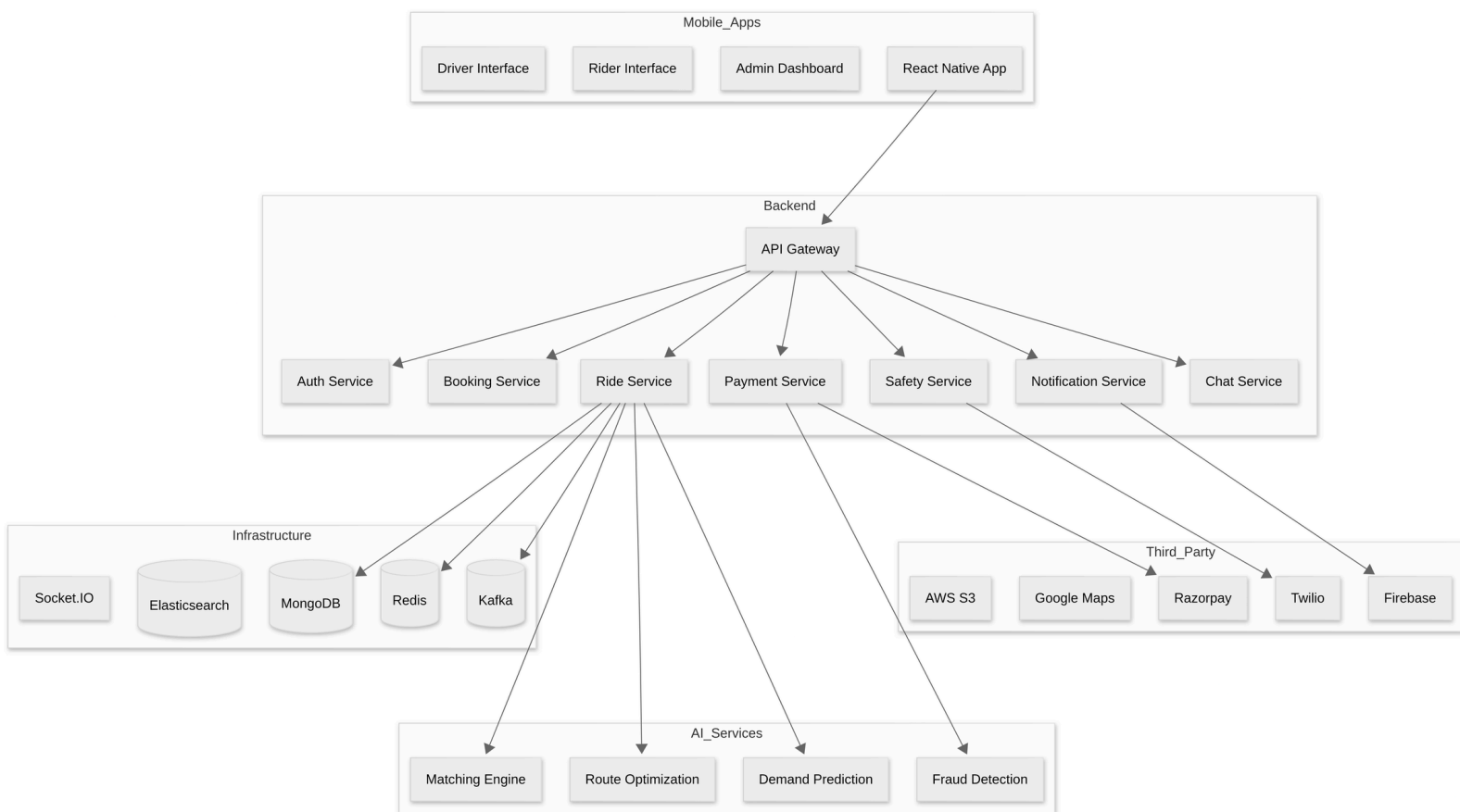
- Car Pooling
- Ride Sharing
- Trip Pooling
- Parcel Pooling
- Real-Time Route Coordination
- Smart Pickup Optimization
- Safety-First Shared Mobility

2. Project Description

The platform enables drivers to publish scheduled rides while passengers search, request, and securely book seats through a real-time intelligent matching system powered by geospatial intelligence and machine learning.

In addition to passenger mobility, Sanchari also supports parcel delivery and logistics pooling, allowing underutilized vehicle space to be used more efficiently for the transportation of lightweight goods.

Built on a scalable microservices-based architecture with real-time communication infrastructure, Sanchari combines AI-based ride matching, predictive demand analysis, route optimization, live tracking, safety monitoring, and secure payment workflows into a unified mobility ecosystem.



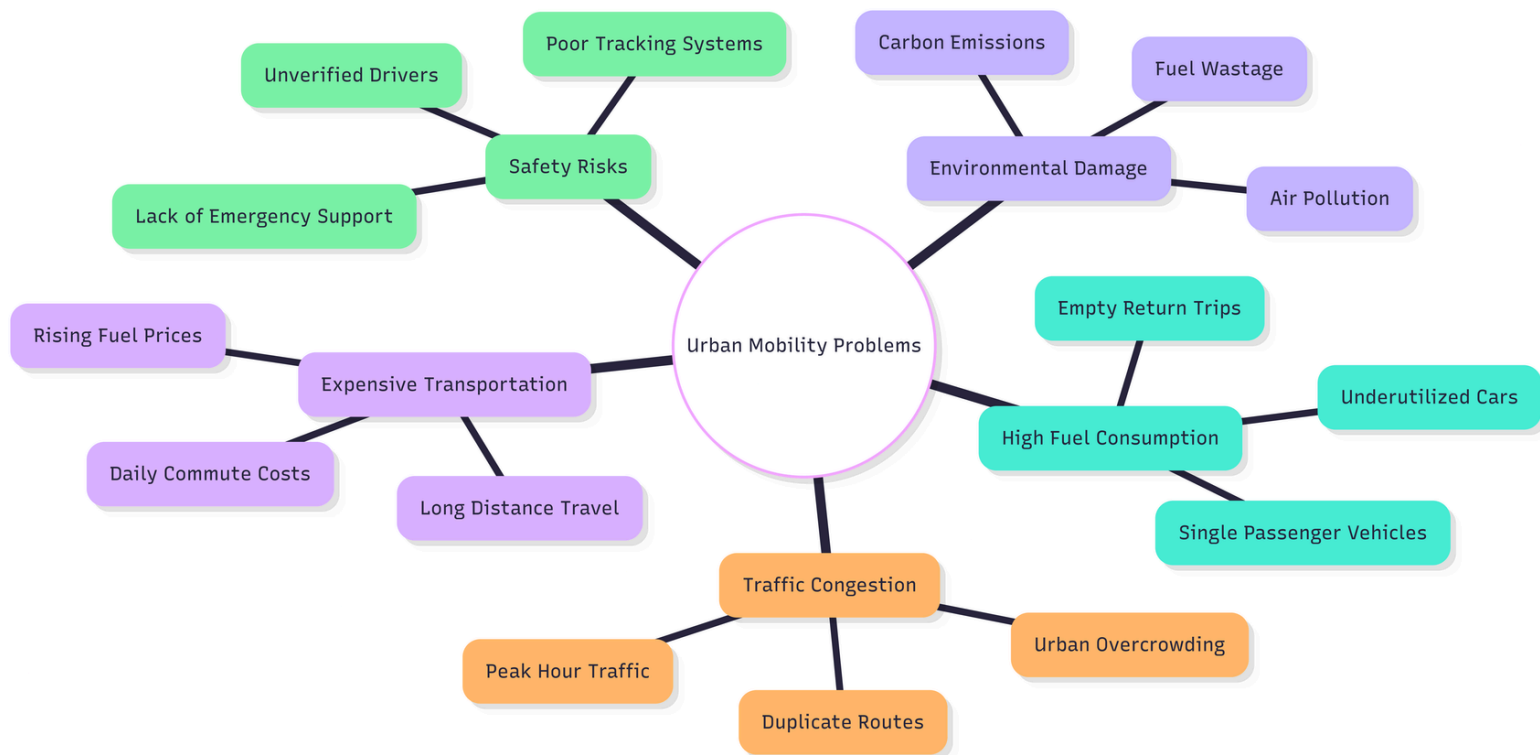
3. Problem Statement

Urban commuters, especially students, office workers, and daily travelers, face increasing transportation expenses, severe traffic congestion, rising fuel prices, and inefficient vehicle utilization.

Most existing ride-hailing systems prioritize instant single-user trips rather than optimized scheduled pooling, leading to increased fuel wastage, empty return trips, and higher commuting costs.

Key Challenges

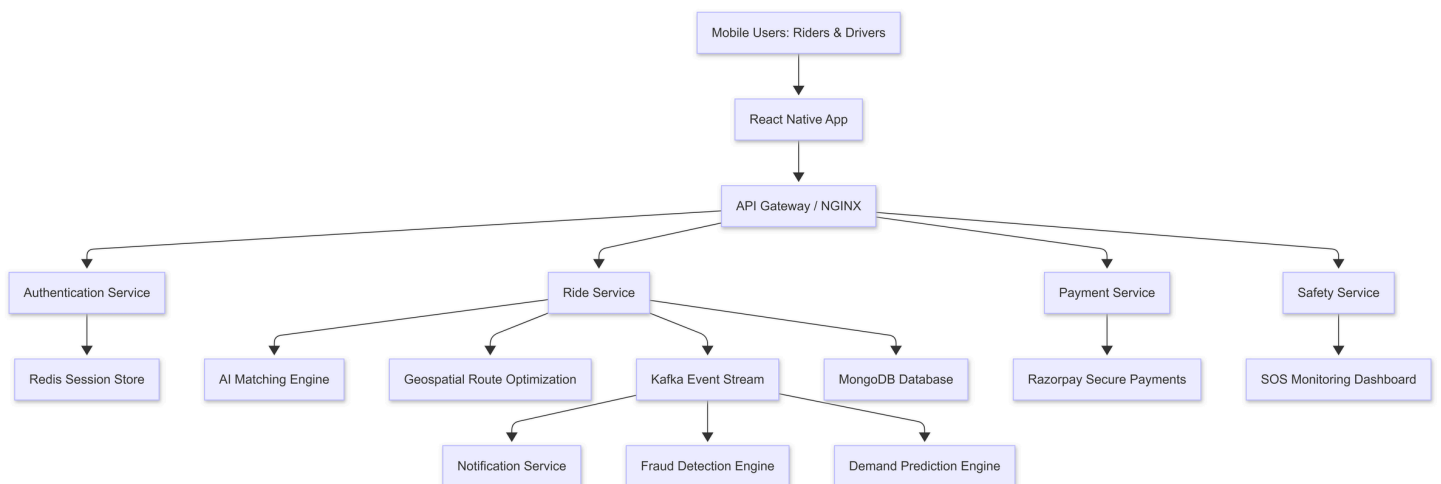
- High fuel consumption
- Traffic congestion
- Underutilized vehicle capacity
- Weak safety systems
- Poor ride compatibility
- Lack of intelligent route optimization
- Fraud and fake booking risks
- Limited women-focused safety features
- Environmental pollution



4. Proposed Solution

Sanchari is a cloud-native AI-powered mobility platform built using React Native, Node.js, MongoDB, Redis, Kafka, Docker, and Socket.IO that enables intelligent ride pooling, real-time communication, predictive demand analytics, geospatial route optimization, secure payment workflows, and scalable distributed transportation infrastructure through a microservices-based architecture.

It is designed as a production-ready mobility ecosystem with event-driven communication, distributed caching, queue-based processing, API gateway routing, secure authentication middleware, and load-balanced backend services to support reliable ride coordination at scale.



Core Platform Objectives

Objective	Purpose
Transportation Optimization	Reduce duplicate trips and improve vehicle utilization
Smart Mobility Infrastructure	Enable AI-driven ride coordination and routing
Safety & Trust Ecosystem	Build safer and verified ride-sharing experiences
Economic Accessibility	Lower commuting costs for users
Environmental Sustainability	Reduce fuel wasatge and emissions

5. Functional Requirements

Authentication

- OTP-based login and registration
- JWT token issuance and refresh workflows
- Role-based access control (RBAC)
- Session validation middleware
- Multi-device session handling

Ride Coordination

- Ride publishing and scheduling
- Seat booking workflows
- Real-time ride synchronization
- Dynamic ride matching
- Smart pickup sequencing
- Route deviation detection

Real-Time Communication

- WebSocket event streaming
- Live GPS tracking
- Real-time ETA synchronization
- In-app messaging system
- SOS event broadcasting
- Push notification workflows

AI & Optimization

- ML-based ride recommendation engine
- Predictive demand analytics
- Fraud detection engine
- Traffic-aware route optimization
- Geospatial ride clustering
- Vehicle Routing Problem (VRP) optimization
- Travelling Salesman Problem (TSP) optimization

Payments

- Razorpay payment integration
- Wallet infrastructure
- Refund workflows
- Driver payout processing
- Secure transaction validation

Admin & Monitoring

- Ride monitoring dashboard
- KYC approval workflows
- Real-time operational analytics
- Fraud monitoring system
- Incident escalation workflows
- Safety compliance monitoring

6. Platform Features

Rider Features

- Phone + OTP Authentication
- Ride Search & Filtering
- Smart AI Ride Matching
- Real-Time Tracking
- In-App Chat
- Ratings & Reviews
- Live Trip Sharing
- Wallet & Payments
- Emergency SOS
- Women-Only Ride Preference
- Safety Check-In Alerts

Driver Features

- Scheduled Ride Creation
- Recurring Ride Scheduling
- Dynamic Pricing Suggestions
- AI Route Optimization
- Smart Pickup Sequencing
- Earnings Dashboard
- KYC Verification
- Vehicle & Document Management

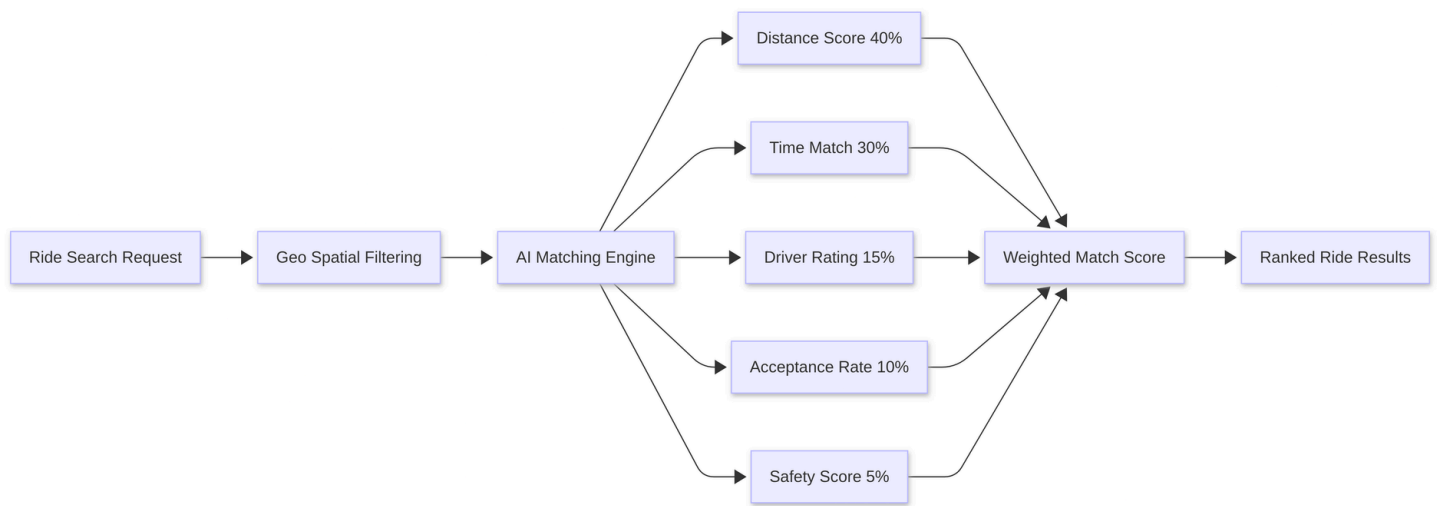
Admin Features

- User Management Dashboard
- Ride Monitoring
- Fraud Detection Monitoring
- SOS Incident Dashboard
- Demand Analytics & Heatmaps
- KYC Approval Workflows
- Real-Time Operational Monitoring

7. AI-Powered Optimization Engine

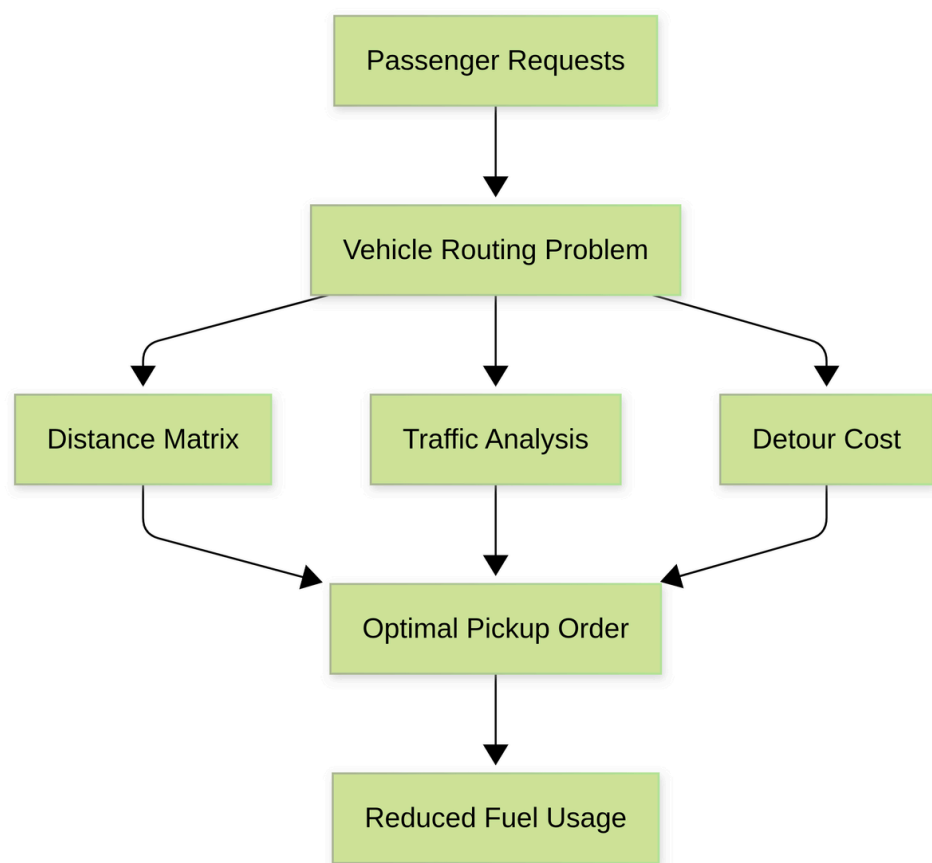
Smart Ride Matching

Uses machine learning-based weighted scoring to match passengers and drivers using route proximity, pickup distance, time compatibility, ratings, acceptance rates, and safety scores.



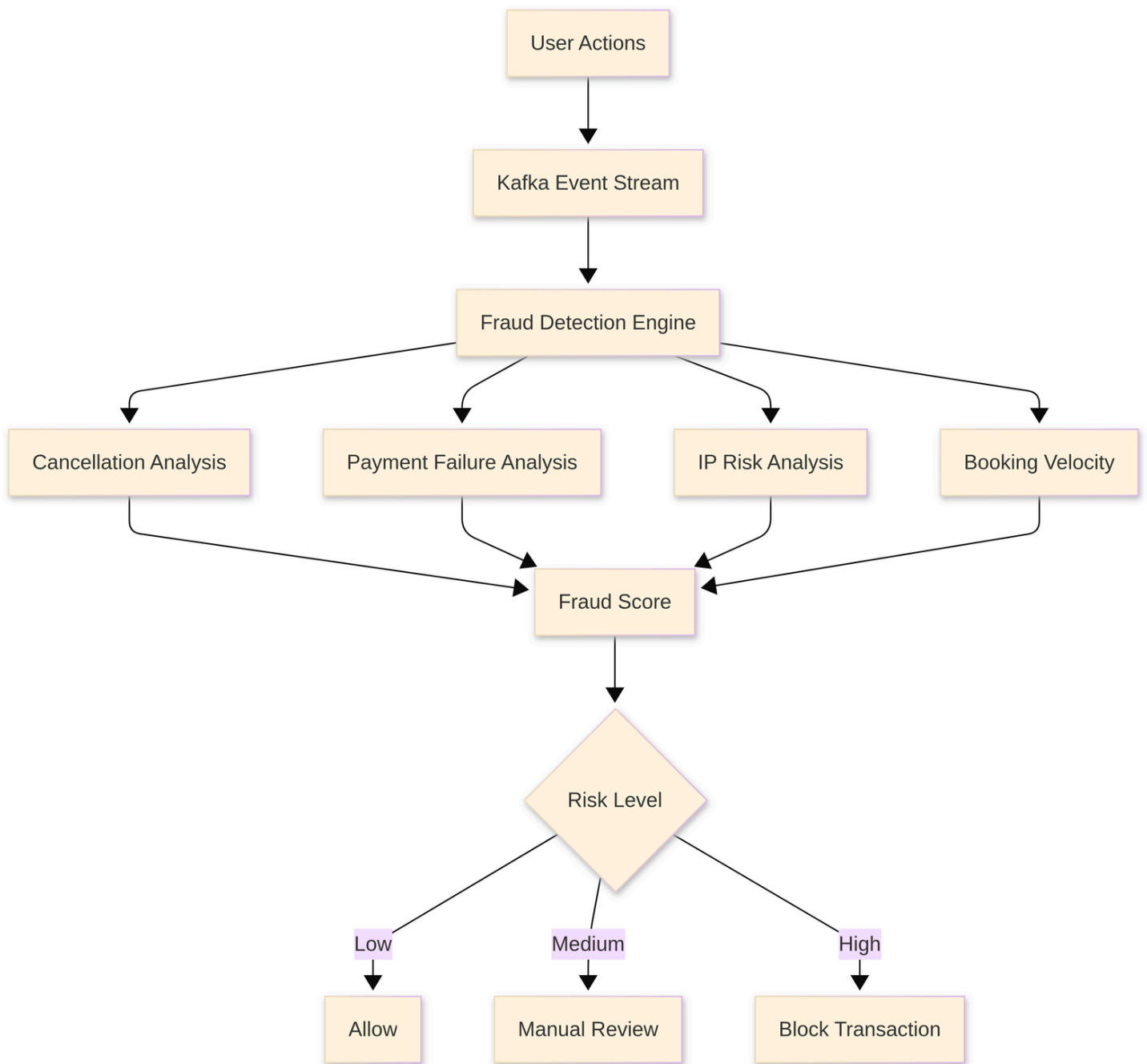
Intelligent Pickup Optimization

Uses the Vehicle Routing Problem (VRP), Travelling Salesman Problem (TSP), and traffic-aware routing to determine optimal pickup sequences while minimizing detours and fuel usage.



Fraud Detection & Demand Prediction

AI models analyze cancellations, fake booking patterns, payment anomalies, and ride demand behavior to improve platform reliability and operational efficiency.



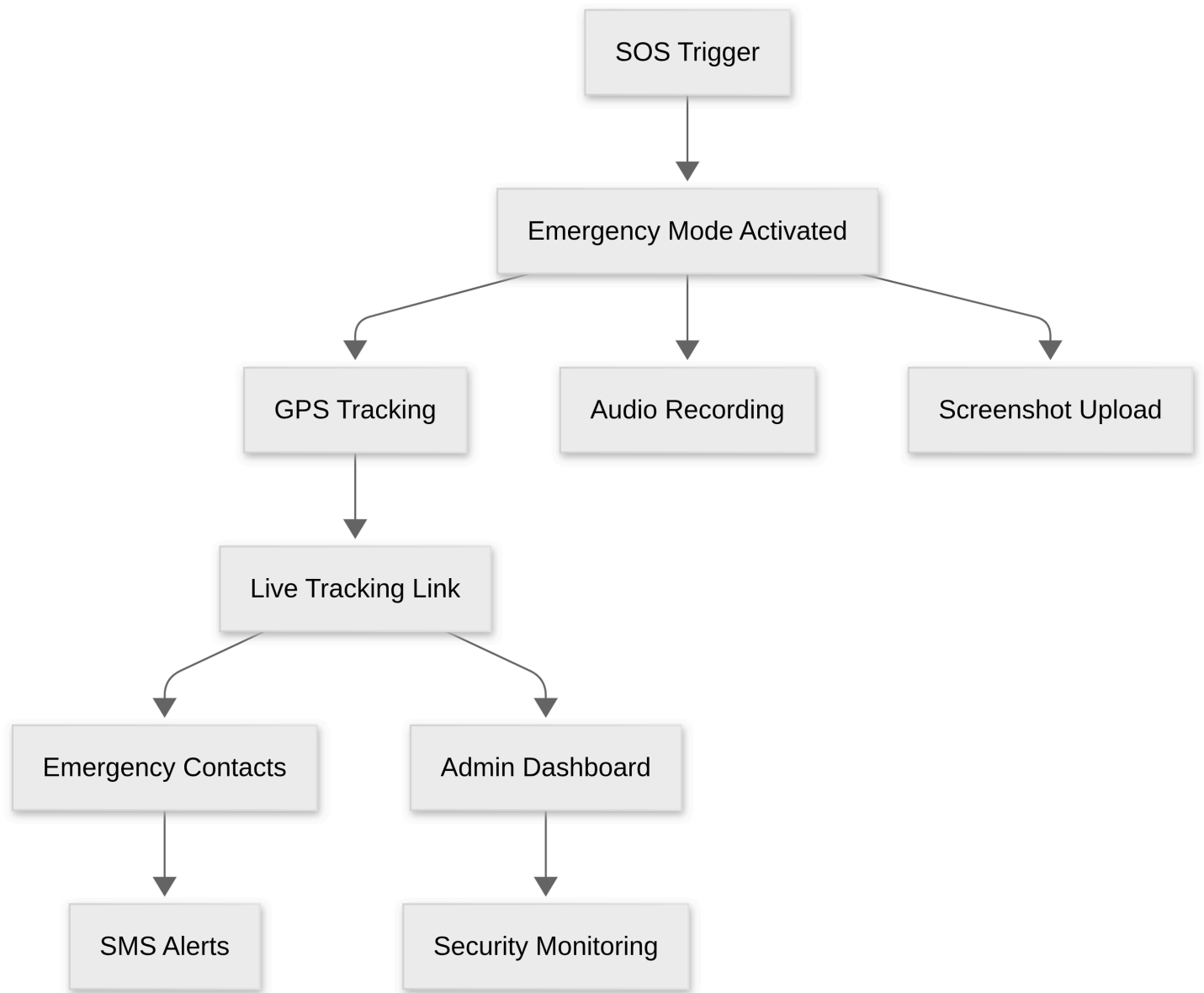
8. Security & Safety Infrastructure

Women's Safety Features

- Women-only ride preferences
- Verified female driver/passenger matching
- Safety-focused route recommendations
- Live ride sharing
- Confidential safety ratings

Emergency Systems

- One-tap SOS activation
- Live GPS emergency tracking
- Emergency contact alerts
- Audio recording triggers
- Real-time incident escalation

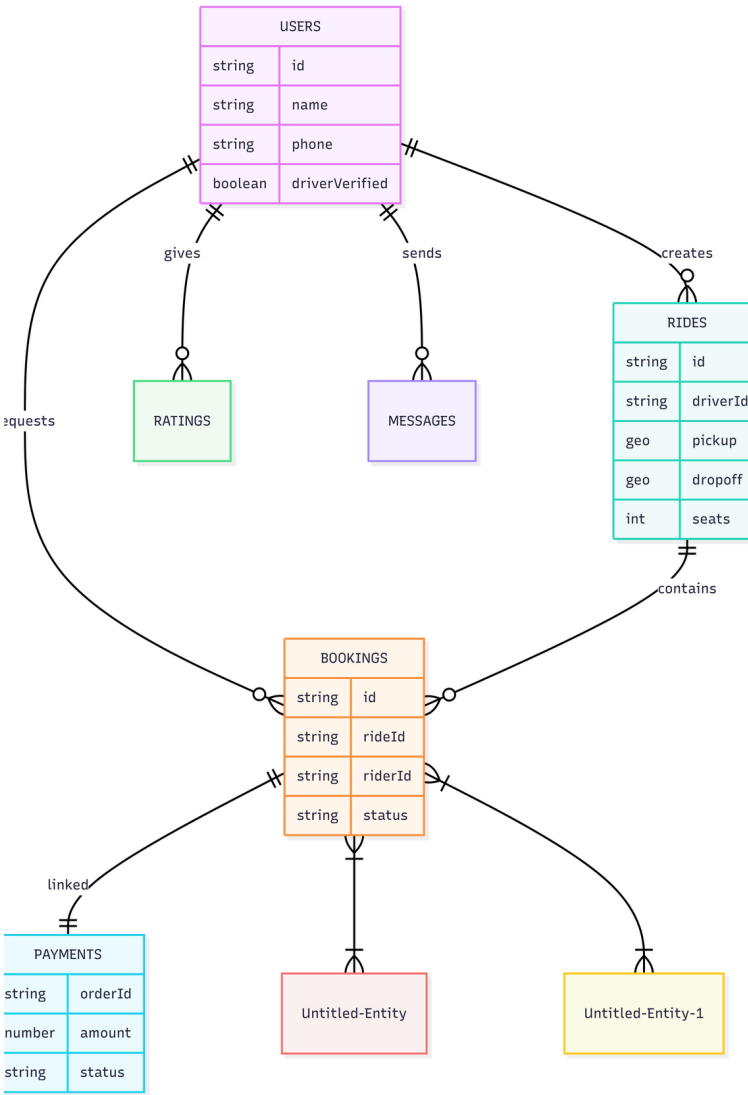


9. Architecture

Technical Architecture

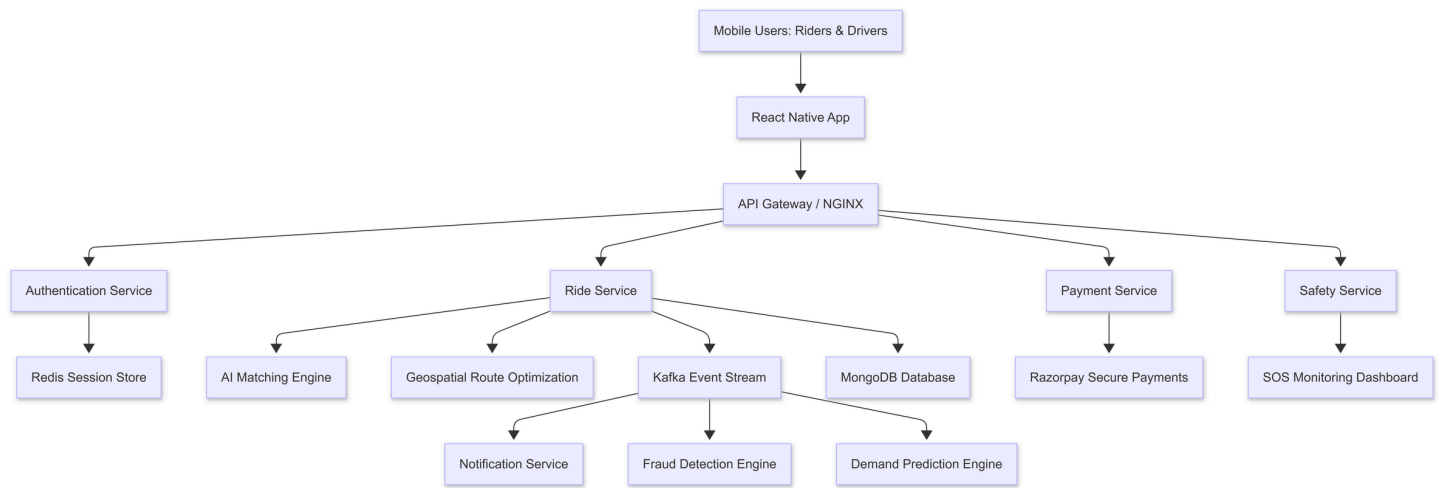
Layer	Technology
Frontend	React Native, TypeScript
Backend	Node.js, Express.js/Fastify
Database	MongoDB
Real-Time	Socket.IO, WebSockets
Cache & Messaging	Redis, Kafka
Authentication	Firebase OTP
Payments	Razorpay
Cloud Infrastructure	AWS S3, Docker

Database Design



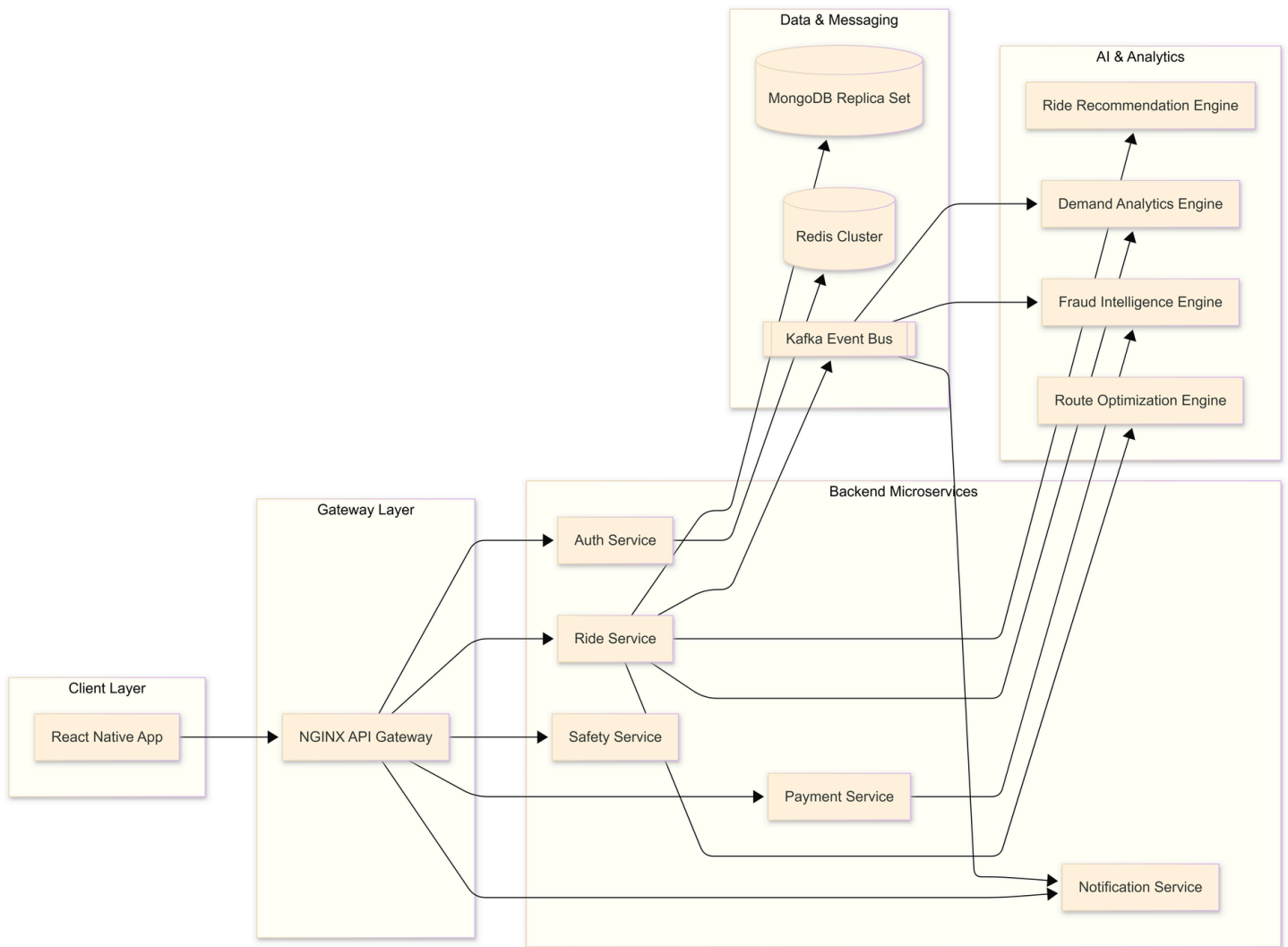
Technical Architecture

- Supports 50K+ concurrent users across rider, driver, and admin workflows.
- API response time target under 300ms for critical ride search and booking requests.
- Sub-100ms WebSocket latency for live location tracking, chat, notifications, and SOS events.
- 99.9% uptime target using high-availability backend services and distributed infrastructure.
- Horizontal scalability through load-balanced backend instances and containerized deployment.
- Fault-tolerant architecture using Kafka event streaming and Redis-backed session management.
- TLS-secured communication and encrypted payment workflows for secure data transmission.
- Automated monitoring, logging, and alerting for system health, performance, and incidents.
- CI/CD-enabled deployment pipeline using GitHub Actions, Docker, and cloud deployment workflows.



Expanded Technical Architecture

Layer	Recommended Technology / Keyword
Frontend	React Native, TypeScript
API Gateway	NGINX Reverse Proxy, RESTful API Routing
Backend	Node.js, Express.js / Fastify, Microservices
Authentication	Firebase OTP, JWT, Session Management, RBAC
Database	MongoDB Replica Set
Real-Time Layer	Socket.IO, WebSockets
Cache & Messaging	Redis Cluster, Kafka Event Streaming
AI Layer	ML Ride Matching, Demand Prediction, Fraud Detection, Route Optimization
Payments	Razorpay, Secure Payment Processing
DevOps	Docker, Kubernetes, GitHub Actions CI/CD
Monitoring	Prometheus, Grafana, Centralized Logging
Cloud Infrastructure	AWS S3, Containerized Deployment, Load Balancing

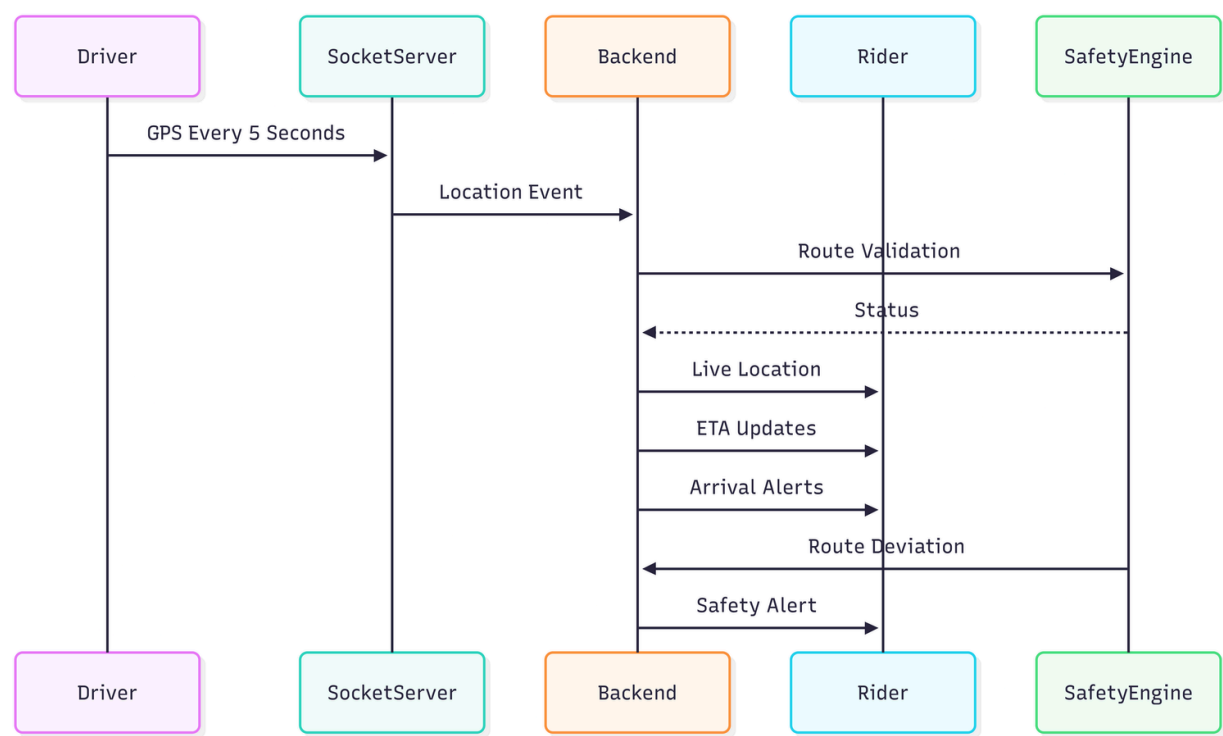


10. Real-Time Distributed Communication Architecture

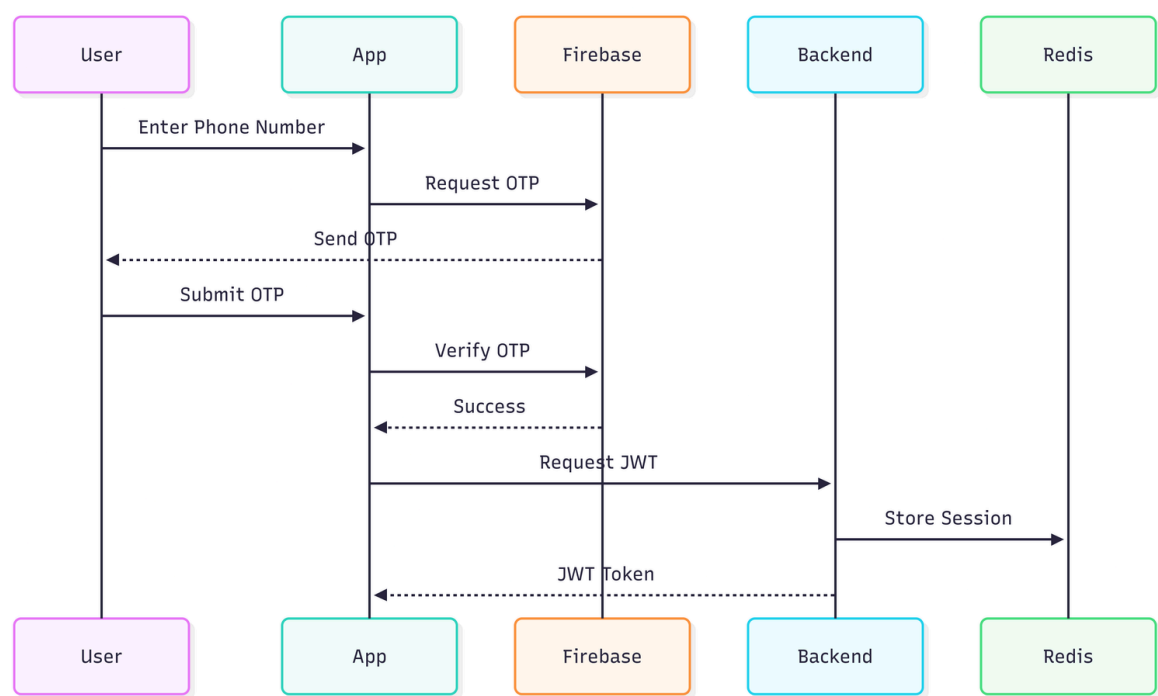
Sanchari uses WebSockets and Socket.IO for live communication between riders, drivers, and backend services.

- Live driver tracking
- Ride ETA updates
- Real-time chat
- Route deviation alerts
- SOS incident broadcasting
- Live notifications

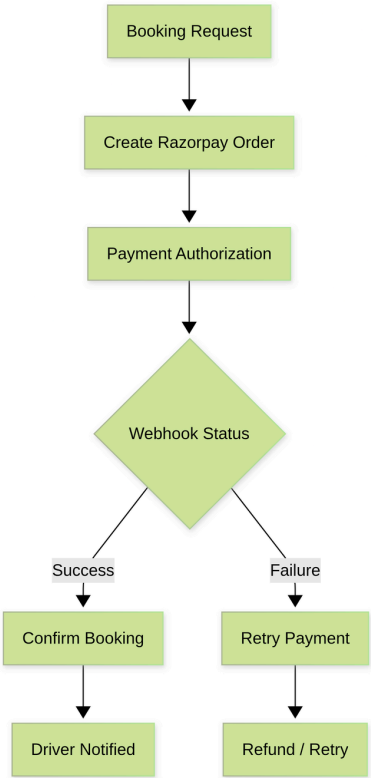
Real-time Tracking



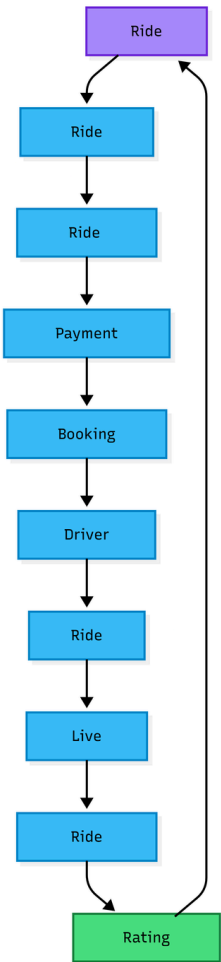
Authentication Workflow



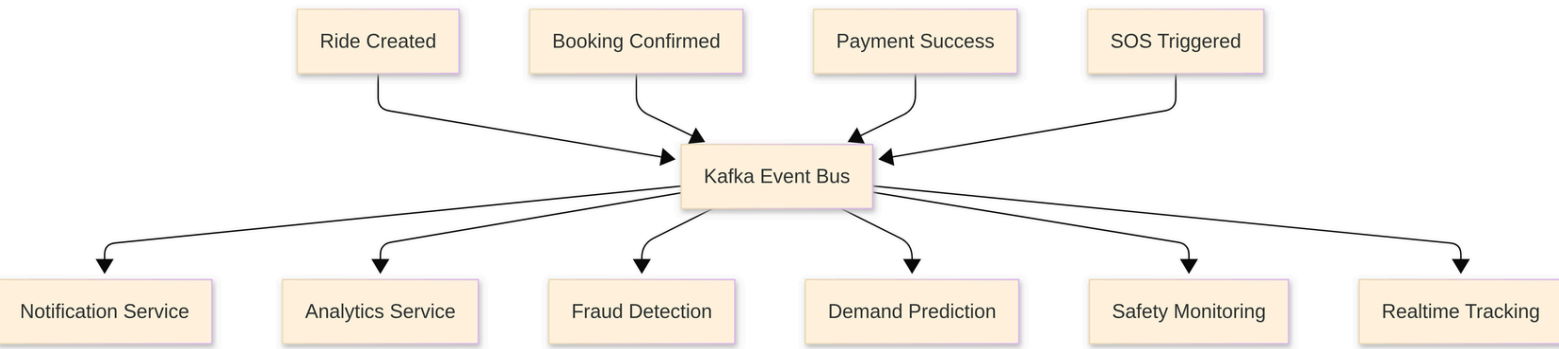
Payment Workflow



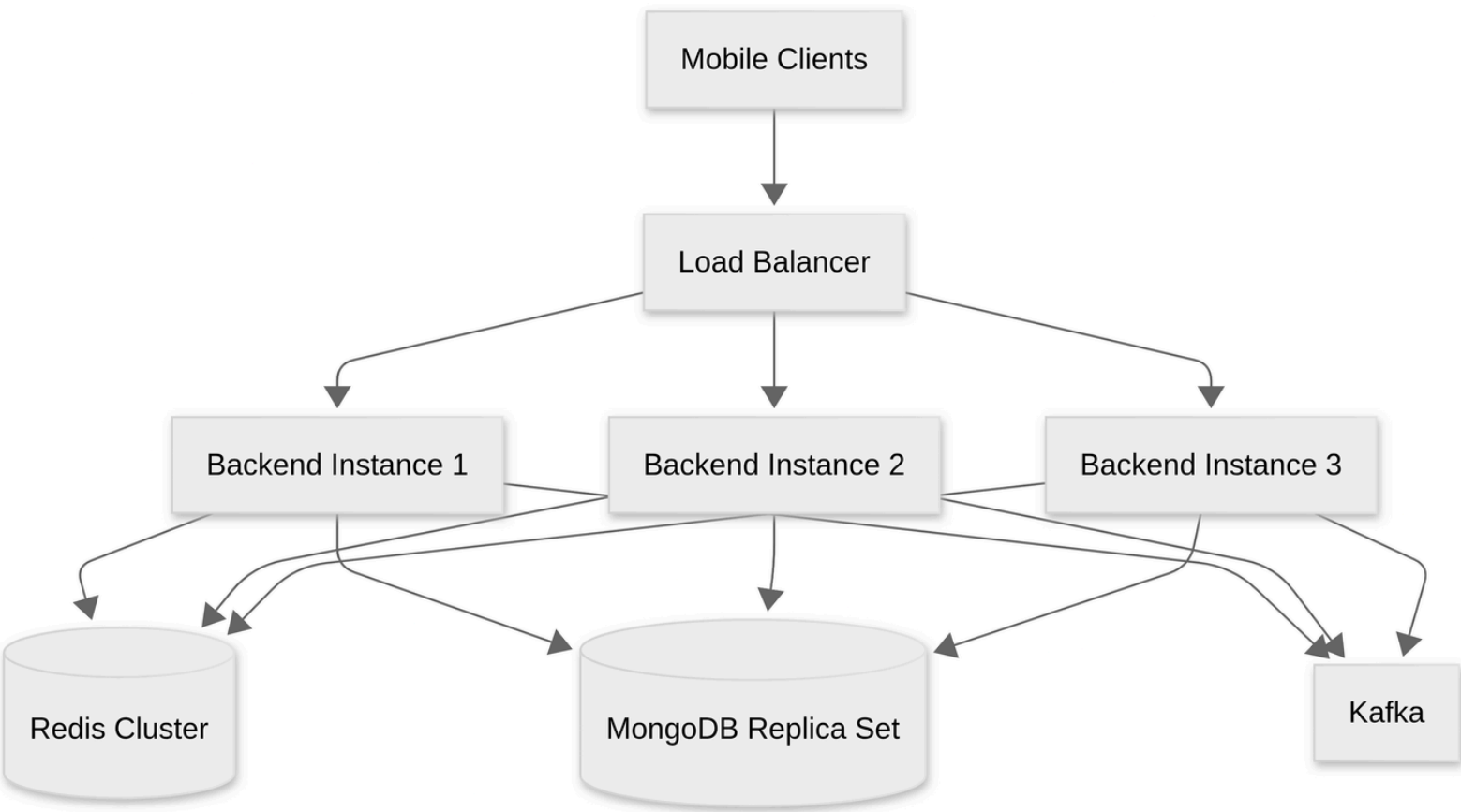
Ride Lifecycle



Event-driven Communication

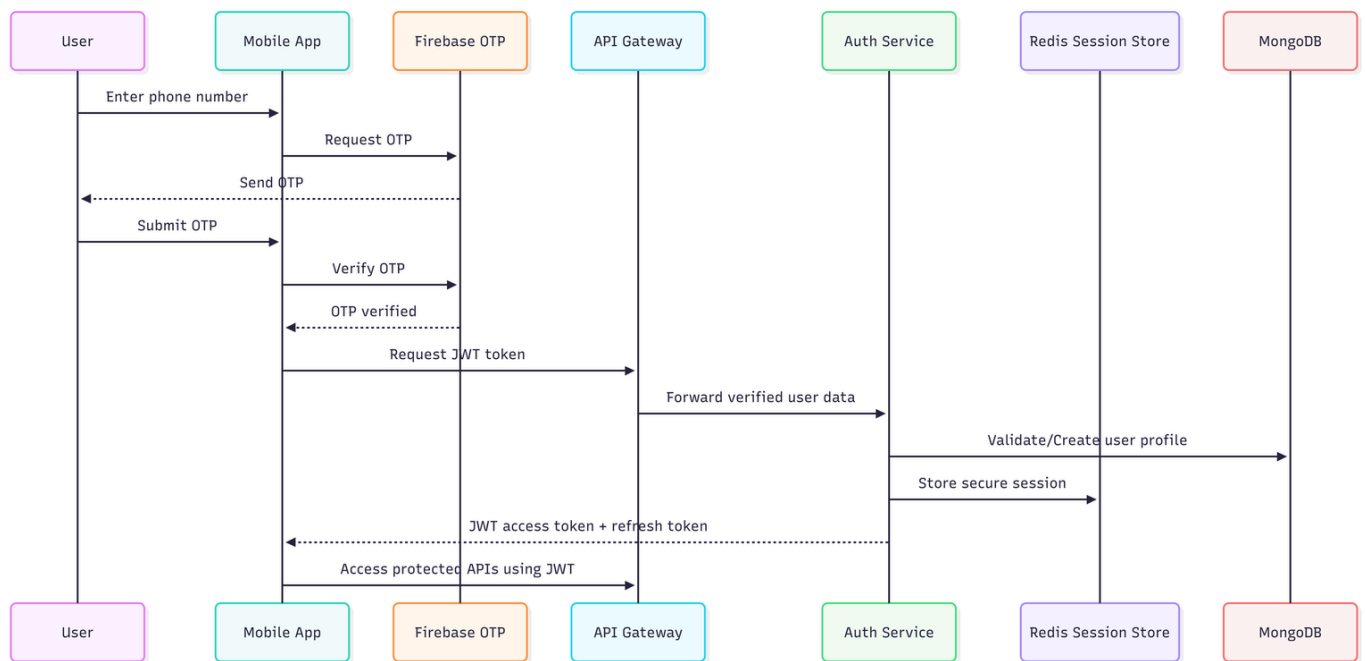


Scalability & Deployment



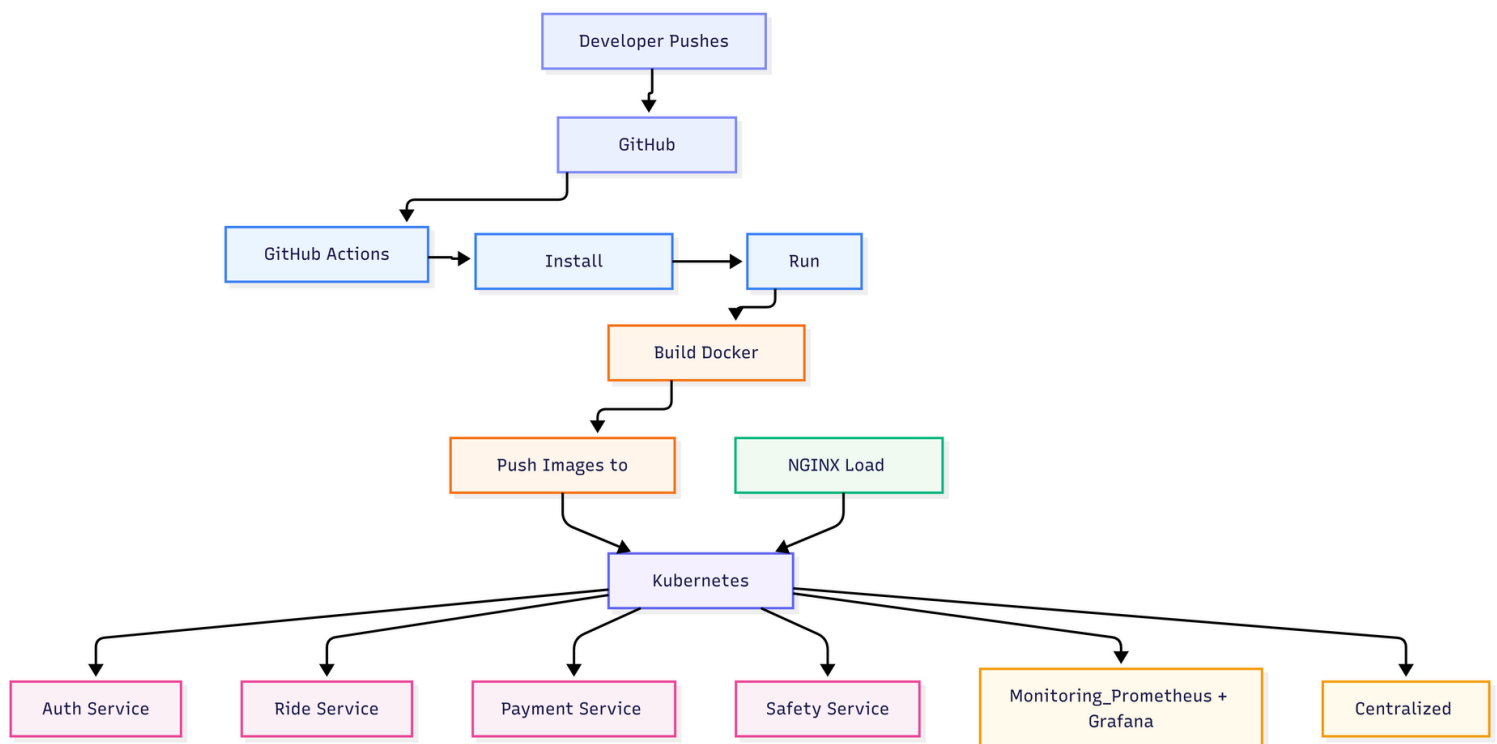
Authentication, Security & Compliance

- JWT-based authentication and secure token refresh mechanism.
- OTP verification workflow for phone-based identity validation.
- Role-based access control for rider, driver, and admin modules.
- Authentication middleware to protect private REST API routes.
- TLS-secured communication between client, gateway, and backend services.
- AES-256 encrypted payment workflow and secure transaction handling.
- Fraud prevention engine for fake bookings, payment anomalies, and suspicious ride behavior.
- Device verification and secure session management using Redis.



Deployment Infrastructure & DevOps Pipeline

- Dockerized microservices deployment for consistent local and cloud environments.
- GitHub Actions CI/CD pipeline for automated build, test, and deployment workflows.
- Kubernetes orchestration for service scaling, service discovery, and container management.
- NGINX reverse proxy for API gateway routing, HTTPS termination, and load balancing.
- AWS S3 integration for document, media, and safety evidence storage.
- Prometheus and Grafana for infrastructure monitoring, metrics, and system health dashboards.
- Centralized logging for debugging ride failures, payment issues, and SOS incidents.



11. Environmental & Social Impact

How Sanchari Helps

- Reduces duplicate vehicle trips
- Improves seat occupancy
- Lowers fuel consumption
- Reduces carbon emissions
- Makes transportation more affordable
- Improves urban mobility access
- Supports sustainable transportation systems



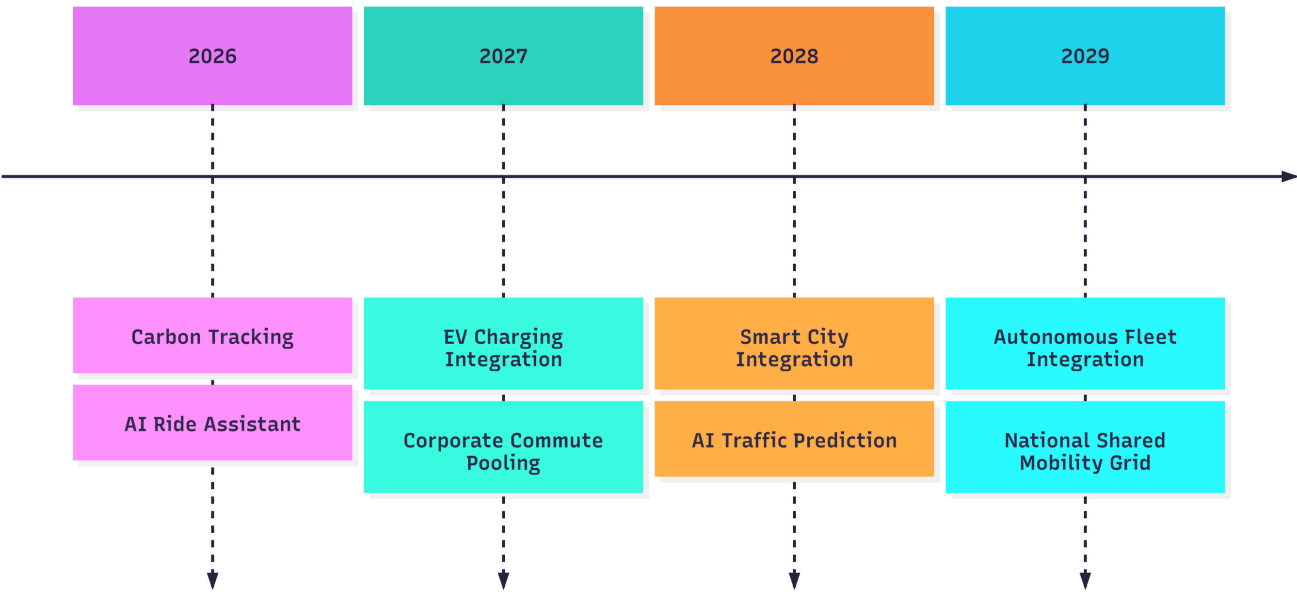
12. Future Scalability & Expansion

Sanchari aims to evolve into a complete smart mobility ecosystem capable of integrating EV transportation systems, smart-city traffic infrastructure, autonomous mobility systems, AI-driven urban transport optimization, and national shared mobility networks.

Future Expansion Areas

- EV charging integration
- Corporate commute pooling
- AI traffic prediction
- Smart-city infrastructure integration
- Autonomous fleet integration
- National mobility grid systems

Future Expansion



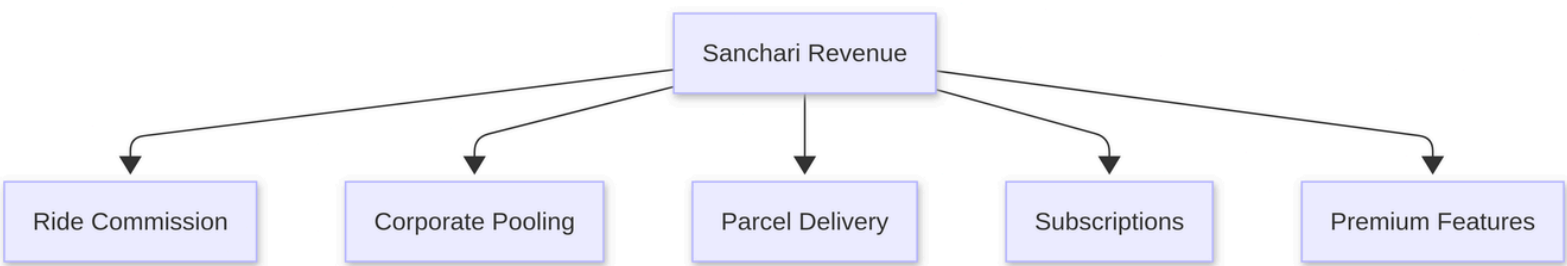
13. Conclusion

Sanchari is more than a ride-sharing app; it is an intelligent transportation ecosystem built for the future, designed to improve urban mobility through AI-powered shared transportation, safety-first infrastructure, and sustainable

mobility optimization.

By combining real-time systems, predictive intelligence, geospatial optimization, and scalable distributed infrastructure, Sanchari creates a reliable, affordable, and environmentally sustainable mobility solution for the next generation of transportation.

Revenue Perspective



Strategic SWOT

SWOT Analysis

Strengths Safety Features Fuel Optimization	Opportunities AI Matching
Weaknesses Regulatory Challenges	Threats User Adoption Barrier